

Unit II: Heuristic Search

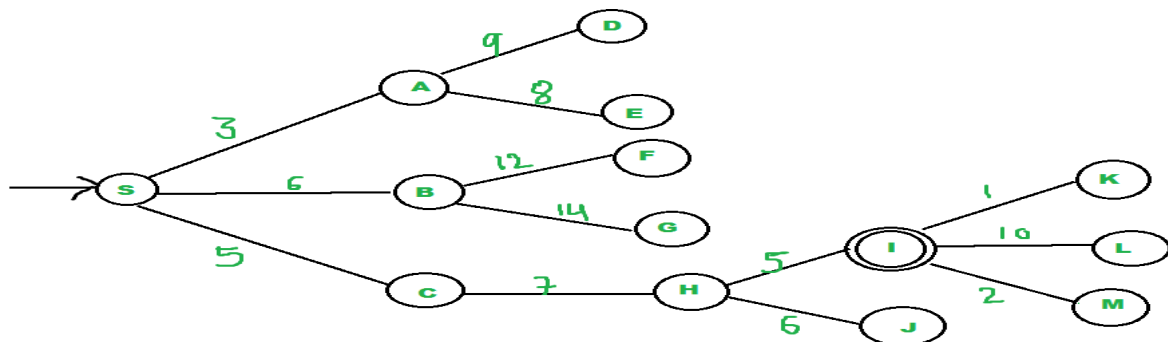
Best First Search

BFS uses the concept of a Priority queue and heuristic search. To search the graph space, the BFS method uses two lists for tracking the traversal. An 'Open' list that keeps track of the current 'immediate' nodes available for traversal and a 'CLOSED' list that keeps track of the nodes already traversed.

Best First Search Algorithm

1. Create 2 empty lists: OPEN and CLOSED
2. Start from the initial node (say N) and put it in the 'ordered' OPEN list
3. Repeat the next steps until the GOAL node is reached
 - If the OPEN list is empty, then EXIT the loop returning 'False'
 - Select the first/top node (say N) in the OPEN list and move it to the CLOSED list. Also, capture the information of the parent node
 - If N is a GOAL node, then move the node to the Closed list and exit the loop returning 'True'. The solution can be found by backtracking the path
 - If N is not the GOAL node, expand node N to generate the 'immediate' next nodes linked to node N and add all those to the OPEN list
 - Reorder the nodes in the OPEN list in ascending order according to an evaluation function $f(n)$

Implementation of Best First Search:



- We start from source "S" and search for goal "I" using given costs and Best First search.
- *pq* initially contains S
 - We remove S from *pq* and process unvisited neighbors of S to *pq*.
 - *pq* now contains {A, C, B} (C is put before B because C has lesser cost)
- We remove A from *pq* and process unvisited neighbors of A to *pq*.
 - *pq* now contains {C, B, E, D}
- We remove C from *pq* and process unvisited neighbors of C to *pq*.
 - *pq* now contains {B, H, E, D}
- We remove B from *pq* and process unvisited neighbors of B to *pq*.

- *pq now contains {H, E, D, F, G}*
- *We remove H from pq.*
- *Since our goal "I" is a neighbor of H, we return.*
-

This algorithm will traverse the shortest path first in the queue. The time complexity of the algorithm is given by $O(n \cdot \log n)$.

- The worst-case time complexity for Best First Search is $O(n \cdot \log n)$ where n is the number of nodes. In the worst case, we may have to visit all nodes before we reach goal. Note that priority queue is implemented using Min(or Max) Heap, and insert and remove operations take $O(\log n)$ time.
- The performance of the algorithm depends on how well the cost or evaluation function is designed.

Hill Climbing Algorithm

- Hill climbing algorithm is a local search algorithm which continuously moves in the direction of increasing elevation/value to find the peak of the mountain or best solution to the problem. It terminates when it reaches a peak value where no neighbor has a higher value.
- Hill climbing algorithm is a technique which is used for optimizing the mathematical problems. One of the widely discussed examples of Hill climbing algorithm is Traveling-salesman Problem in which we need to minimize the distance traveled by the salesman.
- It is also called greedy local search as it only looks to its good immediate neighbor state and not beyond that.
- A node of hill climbing algorithm has two components which are state and value.
- Hill Climbing is mostly used when a good heuristic is available.
- In this algorithm, we don't need to maintain and handle the search tree or graph as it only keeps a single current state.

Features of Hill Climbing:

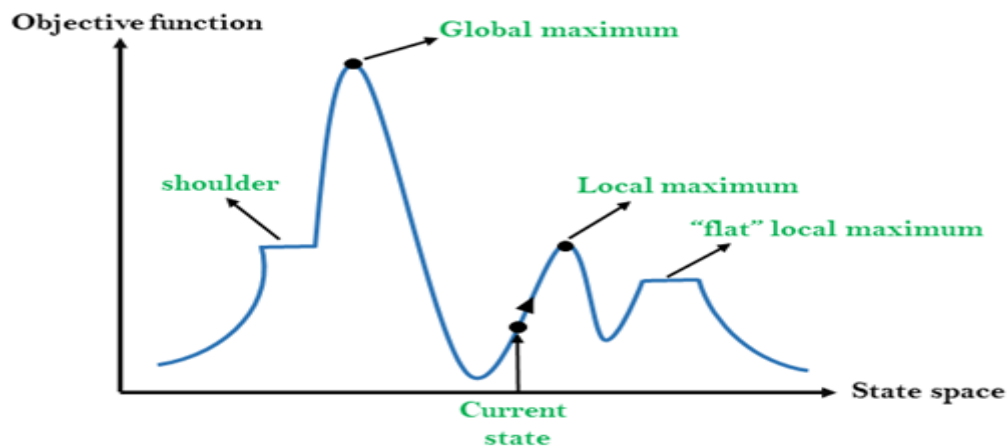
Following are some main features of Hill Climbing Algorithm:

- **Generate and Test variant:** Hill Climbing is the variant of Generate and Test method. The Generate and Test method produce feedback which helps to decide which direction to move in the search space.
- **Greedy approach:** Hill-climbing algorithm search moves in the direction which optimizes the cost.
- **No backtracking:** It does not backtrack the search space, as it does not remember the previous states.

State-space Diagram for Hill Climbing:

The state-space landscape is a graphical representation of the hill-climbing algorithm which is showing a graph between various states of algorithm and Objective function/Cost.

On Y-axis we have taken the function which can be an objective function or cost function, and state-space on the x-axis. If the function on Y-axis is cost then, the goal of search is to find the global minimum and local minimum. If the function of Y-axis is Objective function, then the goal of the search is to find the global maximum and local maximum.



Different regions in the state space landscape:

Local Maximum: Local maximum is a state which is better than its neighbor states, but there is also another state which is higher than it.

Global Maximum: Global maximum is the best possible state of state space landscape. It has the highest value of objective function.

Current state: It is a state in a landscape diagram where an agent is currently present.

Flat local maximum: It is a flat space in the landscape where all the neighbor states of current states have the same value.

Shoulder: It is a plateau region which has an uphill edge.

Hill climbing is neither complete nor optimal, has a time complexity of $O(\infty)$ but a space complexity of $O(b)$.

Problems in different regions in Hill climbing

Hill climbing cannot reach the optimal/best state(global maximum) if it enters any of the following regions :

- **Local maximum:** At a local maximum all neighboring states have a value that is worse than the current state. Since hill-climbing uses a greedy approach, it will

not move to the worse state and terminate itself. The process will end even though a better solution may exist.

To overcome the local maximum problem: Utilize the backtracking technique. Maintain a list of visited states. If the search reaches an undesirable state, it can backtrack to the previous configuration and explore a new path.

- **Plateau:** On the plateau, all neighbors have the same value. Hence, it is not possible to select the best direction.

To overcome plateaus: Make a big jump. Randomly select a state far away from the current state. Chances are that we will land in a non-plateau region.

- **Ridge:** Any point on a ridge can look like a peak because movement in all possible directions is downward. Hence the algorithm stops when it reaches this state.

To overcome Ridge: In this kind of obstacle, use two or more rules before testing. It implies moving in several directions at once.

Beam Search

Beam search is a heuristic search algorithm that explores a graph by expanding the most optimistic node in a limited set. Beam search is an optimization of *best-first search* that reduces its memory requirements.

Best-first search is a graph search that orders all partial solutions according to some heuristic. But in beam search, only a predetermined number of best partial solutions are kept as candidates. Therefore, it is a *greedy* algorithm.

Beam search uses *breadth-first search* to build its search tree. At each level of the tree, it generates all successors of the states at the current level, sorting them in increasing order of heuristic cost. However, it only stores a predetermined number (β), of best states at each level called the *beamwidth*. Only those states are expanded next.

The greater the beam width, the fewer states are pruned. No states are pruned with infinite beam width, and beam search is identical to breadth-first search. The beamwidth bounds the memory required to perform the search. Since a goal state could potentially be pruned, beam search sacrifices completeness (the guarantee that an algorithm will terminate with a solution if one exists). Beam search is not optimal, which means there is no guarantee that it will find the best solution.

In general, beam search returns the first solution found. Once reaching the configured maximum search depth (i.e., translation length), the algorithm will evaluate the solutions found during a search at various depths and return the best one that has the highest probability.

The beam width can either be *fixed* or *variable*. One approach that uses a variable beam width starts with the width at a minimum. If no solution is found, the beam is widened, and the procedure is repeated.

Components of Beam Search

A beam search takes three components as its input:

1. A problem to be solved,
2. A set of heuristic rules for pruning,
3. And a memory with a limited available capacity.

The problem is the problem to be solved, usually represented as a graph, and contains a set of nodes in which one or more of the nodes represents a goal. The set of heuristic rules are rules specific to the problem domain and prune unfavorable nodes from memory regarding the problem domain.

The memory is where the "*beam*" is stored, memory is full, and a node is to be added to the beam, the most costly node will be deleted, such that the memory limit is not exceeded.

Beam Search Algorithm

The following algorithm for a beam search, as a modified best-first search, is adapted from Zhang's 1999:

1. beamSearch(problemSet, ruleSet, memorySize)
2. openMemory = new memory of size memorySize
3. nodeList = problemSet.listOfNodes
4. node = root or initial search node
5. Add node to openMemory;
6. while (node is not a goal node)
7. Delete node from openMemory;
8. Expand node and obtain its children, evaluate those children;
9. If a child node is pruned according to a rule in ruleSet, delete it;
10. Place remaining, non-pruned children into openMemory;
11. If memory is full and has no room for new nodes, remove the worst
12. node, determined by ruleSet, in openMemory;
13. node = the least costly node in openMemory;

Uses of Beam Search

A beam search is most often used to maintain tractability in large systems with insufficient memory to store the entire search tree. For example,

- It has been used in many machine translation systems.
- Each part is processed to select the best translation, and many different ways of translating the words appear.

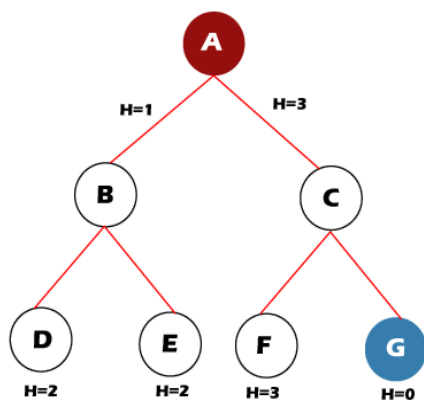
- According to their sentence structures, the top best translations are kept, and the rest are discarded. The translator then evaluates the translations according to a given criterion, choosing the translation which best keeps the goals.
- The first use of a beam search was in the *Harpy Speech Recognition System*, CMU 1976.

Drawbacks of Beam Search

Here is a drawback of the Beam Search with an example:

- In general, the Beam Search Algorithm is not *complete*. Despite these disadvantages, beam search has found success in the practical areas of *speech recognition*, *vision*, *planning*, and *machine learning*.
- The main disadvantages of a beam search are that the search may not result in an optimal goal and may not even reach a goal at all after given unlimited time and memory when there is a path from the start node to the goal node.
- The beam search algorithm terminates for two cases: a required goal node is reached, or a goal node is not reached, and there are no nodes left to be explored.
- A more accurate heuristic function and a larger beam width can improve Beam Search's chances of finding the goal.

For example, let's take the value of $B = 2$ for the tree shown below. So, follow the following steps to find the goal node.



Step 1: OPEN= {A}

Step 2: OPEN= {B, C}

Step 3: OPEN= {D, E}

Step 4: OPEN= {E}

Step 5: OPEN= { }

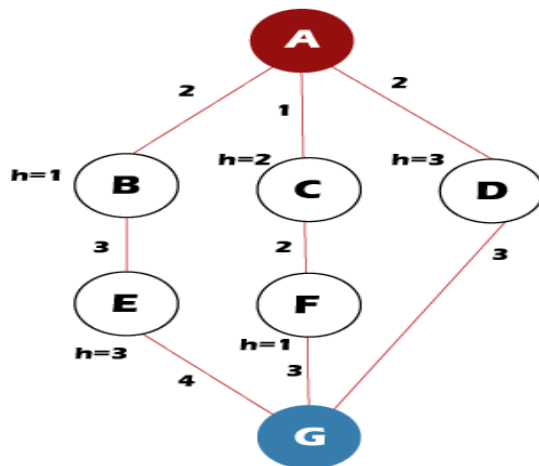
The open set becomes empty without finding the goal node.

Beam Search Optimality

The Beam Search algorithm is not complete in some cases. Therefore it is also not guaranteed to be optimal. It can happen because of these reasons:

- The beam width and an inaccurate heuristic function may cause the algorithm to miss expanding the shortest path.
- A more precise heuristic function and a larger beam width can make Beam Search more likely to find the optimal path to the goal.

For example, we have a tree with heuristic values shown below:



Follow the following steps to find the path for the goal node.

Step 1: OPEN= {A}

Step 2: OPEN= {B, C}

Step 3: OPEN= {C, E}

Step 4: OPEN= {F, E}

Step 5: OPEN= {G, E}

Step 6: Found the goal node {G}, now stop.

Path: A-> C-> F-> G

But the **Optimal Path** is A-> D-> G

Time Complexity of Beam Search

The time complexity of the Beam Search algorithm depends on the following things, such as:

- The accuracy of the heuristic function.
- In the worst case, the heuristic function leads Beam Search to the deepest level in the search tree.
- The worst-case time = $O(B*m)$

B is the beam width, and m is the maximum depth of any path in the search tree.

Space Complexity of Beam Search

The space complexity of the Beam Search algorithm depends on the following things, such as:

- Beam Search's memory consumption is its most desirable trait.
- Since the algorithm only stores B nodes at each level in the search tree.
- The worst-case space complexity = $O(B*m)$

B is the beam width, and m is the maximum depth of any path in the search tree.

- This linear memory consumption allows Beam Search to probe very deeply into large search spaces and potentially find solutions that other algorithms cannot reach.

What is TABU Search?

Tabu search (TS) is an iterative neighborhood search algorithm, where the neighborhood changes dynamically. Tabu search enhances local search by avoiding points in the search space which are already visited. By avoiding already visited points, loops in search space are avoided and local optima can be escaped.

The main feature of Tabu Search is the use of explicit memory, with two goals:

- To prevent the search from revisiting previously visited solutions.
- To explore the unvisited areas of the solution space.

Randomized Search: Simulated Annealing

Random search is a technique where random combinations of the hyperparameters are used to find the best solution for the built model. It is similar to grid search, and yet it has proven to yield better results comparatively. The drawback of random search is that it yields high variance during computing. Since the selection of parameters is completely random; and since no intelligence is used to sample these combinations, luck plays its part.